Labs 2-5 and Midterm Defense (Replicated Counter Service)

Daniil Muntyan

1. Requirements under test

Our service is a fault-tolerant distributed counter built with gRPC in Python. It supports Increment and Get operations with majority quorum agreement (2 out of 3 replicas) and Lamport logical clocks for causal ordering.

Task	Requirement	Verified By
A2	At-most-once RPC semantics, retries, duplicate suppression	test_increment_applies_delta test_duplicate_key_not_reapplied test_retry_after_timeout_is_safe
A3	Thread safety and lock synchronization under concurrent clients	test_concurrent_increments_exact
B1	Lamport logical clocks on requests and responses	Captured trace log (logs/trace.log)
C1	Three replicas with 2-of-3 majority quorum commit	test_majority_commit_two_acks test_no_commit_below_majority test_replicas_converge

RPC Semantics:

Our system guarantees **at-most-once** execution semantics for mutations. Every client action generates a unique idempotency_key (UUID v4) that is preserved across all retry attempts. The server tracks processed keys in an in-memory _seen hash map protected by a shared threading.Lock(). When a retry request arrives with an existing key, the server bypasses the counter mutation, returns the cached result, and marks was_duplicate = True.

Task C1 Discussion:

Partial write anomaly: If a client request reaches only 1 replica while the other 2 replicas are down, the quorum threshold is not met ($1 < 2$), and the client receives a failure (QuorumError). However, that single replica has already modified its counter state in memory. A true consensus protocol (like Raft) eliminates this anomaly by appending writes to an uncommitted write-ahead log first, only applying them to the state machine once a quorum has confirmed the log entry.

Get consistency risk: Performing a Get query on a single random replica carries a stale read risk: if that replica was offline during the most recent quorum write, it will return an outdated counter value. Strict consistency would require quorum reads ($R + W > N$).

2. Test environment

Operating System: Windows 11 (64-bit)
Python Runtime: 3.14.7 (Virtual Environment .venv)
Packages: grpcio (1.84.0), grpcio-tools (1.84.0), protobuf (7.36.2), pytest (9.1.1)
Hardware: Standard multi-core x86_64 PC, 16 GB RAM

3. Test-case table

All 11 automated tests follow the strict Arrange-Act-Assert pattern implemented via pytest.

ID	Purpose	Expected result	Actual result	Pass/Fail
1	Single increment	Value increases by delta (+5, then +3)	Value is 8	PASS
2	Idempotency check	Same key returns was_duplicate=True	Value stays 5, duplicate = True	PASS
3	Concurrency (Task A3)	2 clients * 1,000 adds = 2,000	Value is exactly 2000	PASS
4	Missing key query	Querying uninitialized counter	Returns found=False without crash	PASS
5	Retry after timeout	Slow replica, retry moves state once	Counter moves only once (value = 10)	PASS
6	Quorum with 1 dead node	Write succeeds with 2/3 acks	Acked = 2, value committed	PASS
7	Below majority (2 dead)	Write fails when quorum is lost	QuorumError raised, write rejected	PASS
8	State convergence	All 3 replicas hold identical state	Replicas A, B, C all hold value 20	PASS
9	Crash during request	Surviving 2 replicas commit write	Success = True, no unhandled errors	PASS
10	Duplicate network call	Second call returns duplicate flag	Counter remains static (15)	PASS
11	Node latency > 2.0s	Timeout triggers retry, state updates once	Success = True, value moves once (42)	PASS

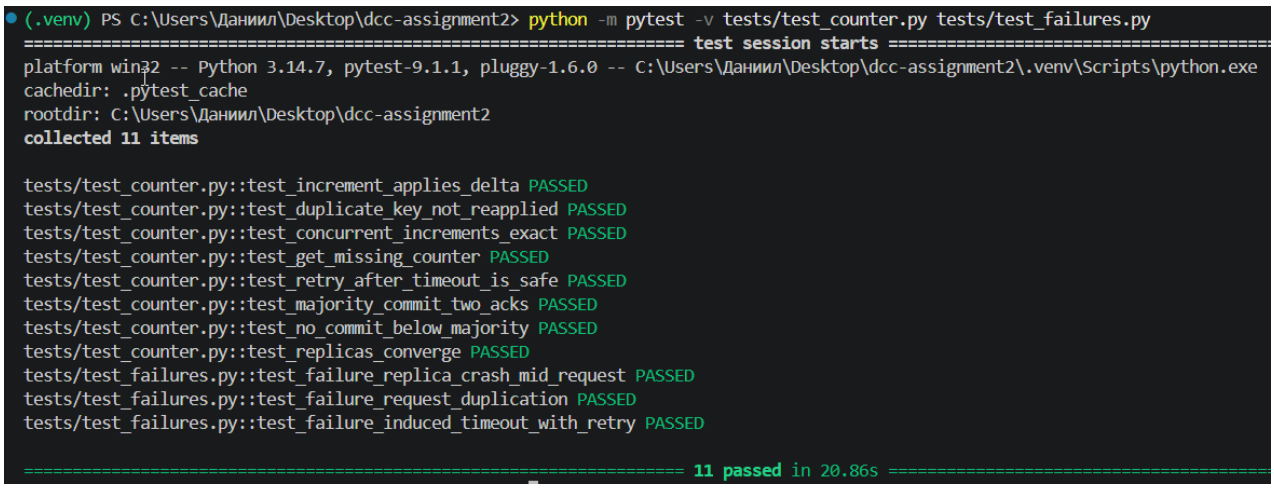


Figure 1. Automated test suite output (11/11 passed).

4. Failure-injection results

Replica crash mid-request:

During an active write request, replica-C was forcefully killed via proc.kill(). The client transparently gathered confirmations from the two remaining nodes (replica-A and replica-B), confirmed the write (2 >= 2), and returned successfully without throwing unhandled exceptions to the caller.

Request duplication:

The same logical operation with key same-operation-key was submitted twice to the cluster. The first call incremented the counter by 15. The second call returned was_duplicate = True, leaving the counter strictly at 15.

Induced timeout with retry:

A replica was initialized with an artificial delay of 2.5 seconds (--fault delay-once), exceeding the client's 2.0s timeout deadline. The client encountered DEADLINE_EXCEEDED, performed exponential backoff, and retried with the identical idempotency key. The retry succeeded and the counter advanced exactly once (value = 42).

5. Performance table and interpretation (Task C4)

We executed 2,000 requests per configuration using perf_benchmark.py. The 95th percentile (p95) was computed by sorting latencies and indexing at 0.95 * n.

Configuration	Median latency (ms)	p95 latency (ms)	Requests
Single replica, 1 client	0.73	1.13	2000
Single replica, 16 clients	5.27	6.40	2000
Quorum (3 replicas), 1 client	1.15	1.64	2000
Quorum (3 replicas), 16 clients	11.19	15.07	2000

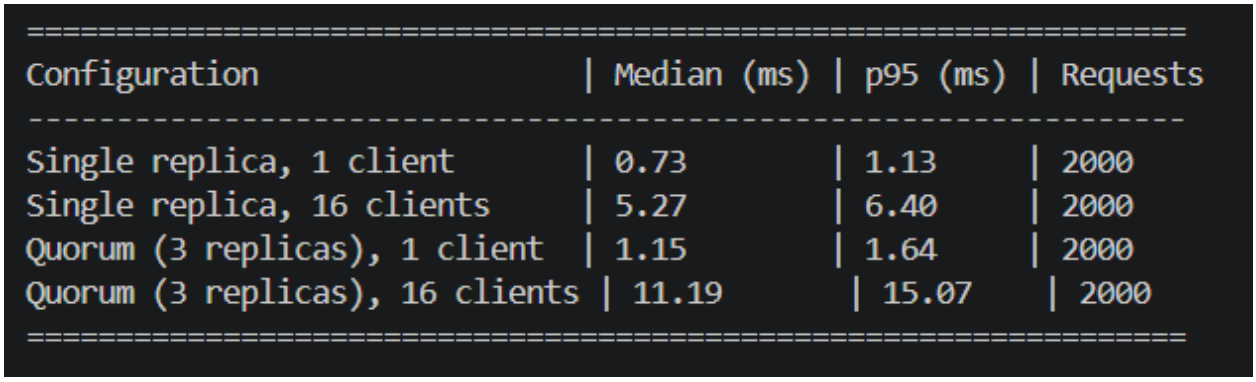


Figure 2. Performance benchmark table by python perf_benchmark.py

Interpretation:

The quorum replication path adds expected overhead compared to the single-replica setup: median latency increases from 0.73 ms to 1.15 ms for 1 client, and from 5.27 ms to 11.19 ms under 16 concurrent clients. This additional delay stems from:

Network Fan-Out: The client dispatches requests concurrently to 3 endpoints and must await the second response before completing.

Lock & Thread Contention: Under 16 concurrent clients, contention on the server's internal threading.Lock() and context-switching overhead increases latency variance.

This latency (~11 ms median, ~15 ms p95) is completely acceptable for a social network like counter, as counter increments run asynchronously in the background and users will not perceive an extra 10 ms delay.

6. Logical Time and Causal Ordering Analysis (Task B2)

From the captured execution trace in logs/trace.log, we construct the space-time happened-before diagram showing message transmissions and causal progressions.

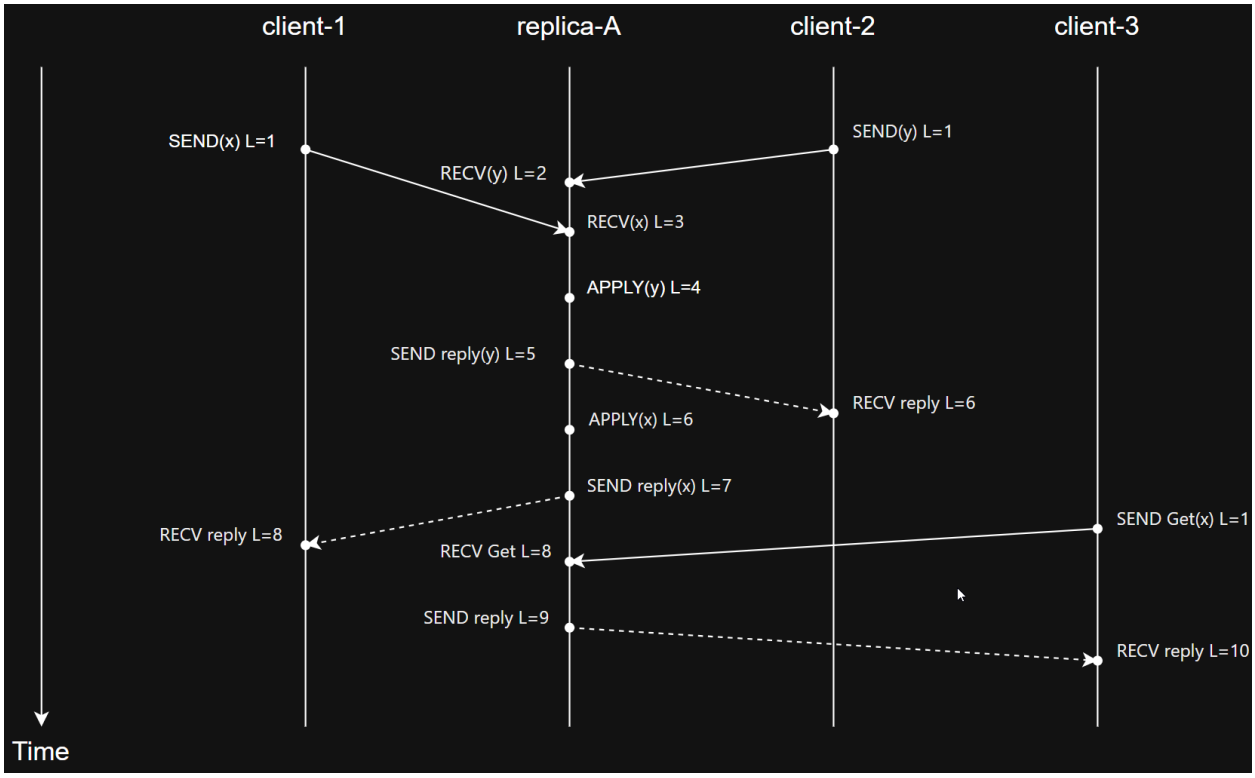


Figure 3. Happened-before causal diagram (Task B2 trace).

Causally Ordered Event Pairs:

Pair 1: [client-1] SEND Increment(x, 1) L=1 => [replica-A] RECV Increment(x, 1) L=3

Proof: Network message edge (send happens-before receive).

Pair 2: [replica-A] RECV Increment(x, 1) L=3 => [replica-A] APPLY counter=x => 1 L=6

Proof: Sequential program execution order on replica-A.

Concurrent Events:

[client-1] SEND Increment(x, 1) L=1 and [client-2] SEND Increment(y, 1) L=1.

Explanation: The two clients issued independent increment requests without prior communication. Even if one client had a higher Lamport timestamp due to previous local ticks, scalar clocks only guarantee: if a=> b, then L(a) < L(b). The converse does not hold, meaning L(a) < L(b) does not prove that a happened before b.

Limitation of Lamport Clocks vs Vector Clocks:

Lamport clocks impose an arbitrary total order that hides concurrency. Vector clocks eliminate this limitation by tracking causal event histories for every node independently, allowing true bidirectional detection of causal dependencies and concurrent conflicts (V(a) || V(b)).

7. Known limitations

Data lost on restart (In-Memory only): Counters and deduplication keys are stored strictly in memory (_values, _seen). If all replicas crash and restart, state is completely lost. A production system must persist writes to a Write-Ahead Log (WAL) on disk.

Stale reads on Get: The Get operation queries only one replica. If that replica missed the latest update due to temporary network partition, it returns stale data. Strong consistency requires reading from a quorum (R + W > N).

Partial write anomaly on quorum failure: If 1 replica applies a write but the remaining 2 replicas fail, the client receives a failure, yet the surviving replica retains the increment without a rollback mechanism.

8. Reflection questions

1. Request lost vs reply lost:

A timeout only indicates that the client-side timer expired. The client cannot determine whether the request was lost before reaching the server or if the server applied the write and only the reply was lost. Our idempotency design makes this harmless: the client retries with the same idempotency_key (UUID). If the write was already applied, the server returns the cached result without re-incrementing the counter.

2. Concurrent events with different Lamport values:

Lamport clocks increment monotonically on local events. If node A performs 5 local actions, its clock advances to L=5. If node B was idle, its clock remains at L=1. If B now issues an event, L(B) = 1 < L(A) = 5, but B did not happen first in physical time. They are concurrent because neither node observed the other's state.

3. Fault tolerance and Safety vs Liveness:

With 3 replicas and a majority quorum of 2, the system tolerates 1 node crash while preserving both Safety and Liveness. If 2 replicas crash, the system loses Liveness (it cannot commit writes and returns errors), but it preserves Safety (it prevents split-brain and never accepts uncommitted, divergent writes).

4. Surviving restarts:

To survive crashes without losing committed writes, each replica must append mutations and deduplication keys to an append-only Write-Ahead Log (WAL) on disk before responding. On boot, the server replays the WAL to reconstruct state. This directly anticipates the Replicated Log mechanism in Raft (Week 6).

Appendix A: Captured Event Trace (logs/trace.log)

[client-1] SEND Increment(counter=x, delta=1) L=1
[client-2] SEND Increment(counter=y, delta=1) L=1
[replica-A] RECV Increment(counter=y, delta=1) L=2 (received L=1)
[replica-A] RECV Increment(counter=x, delta=1) L=3 (received L=1)
[replica-A] APPLY counter=y -> 1 L=4
[replica-A] SEND IncrementReply(new_value=1) L=5
[client-2] RECV IncrementReply(new_value=1) L=6 (received L=5)
[replica-A] APPLY counter=x -> 1 L=6
[replica-A] SEND IncrementReply(new_value=1) L=7
[client-1] RECV IncrementReply(new_value=1) L=8 (received L=7)
[client-3] SEND Get(counter=x) L=1
[replica-A] RECV Get(counter=x) L=8 (received L=1)
[replica-A] SEND GetReply(value=1) L=9
[client-3] RECV GetReply(value=1) L=10 (received L=9)
[client-2] SEND Increment(counter=y, delta=1) L=7
[client-1] SEND Increment(counter=x, delta=1) L=9
[replica-A] RECV Increment(counter=x, delta=1) L=10 (received L=9)
[replica-A] APPLY counter=x -> 2 L=11
[replica-A] SEND IncrementReply(new_value=2) L=12
[client-1] RECV IncrementReply(new_value=2) L=13 (received L=12)
[replica-A] RECV Increment(counter=y, delta=1) L=13 (received L=7)
[replica-A] APPLY counter=y -> 2 L=14
[replica-A] SEND IncrementReply(new_value=2) L=15
[client-2] RECV IncrementReply(new_value=2) L=16 (received L=15)
[client-3] SEND Get(counter=y) L=11
[replica-A] RECV Get(counter=y) L=16 (received L=11)
[replica-A] SEND GetReply(value=2) L=17
[client-3] RECV GetReply(value=2) L=18 (received L=17)

Appendix B: Python Environment (pip freeze)

colorama==0.4.6
grpcio==1.84.0
grpcio-tools==1.84.0
iniconfig==2.3.0
packaging==26.3
pluggy==1.6.0
protobuf==7.36.2
Pygments==2.21.0
pytest==9.1.1
setuptools==84.0.0
typing_extensions==4.16.0